Chapter 12 — Testing React

A test is a small program that uses your code and checks the result, so a computer — not you clicking around — notices when something breaks. The most useful React tests **render the component, interact with it as a user would, and check what the user would see**. That is the idea behind **React Testing Library** (RTL), the standard tool for component tests. General testing ideas — `node:test` (Node's built-in runner), assertions (checks that fail the test when false), test doubles and mocking (fakes that replace real parts) — are in the JavaScript Handbook (Chapter 16).

Code: `code/`. Every file is a **runnable test file**: React 19.3, React Testing Library 16.3, `@testing-library/user-event` 14.6 and jsdom (a pure-JavaScript imitation of the browser DOM for Node.js), run by `node:test` with `node:assert/strict`. Output is Node's `spec` reporter (`node --test-reporter=spec file.mjs`); times vary per run, and summary lines for suites, cancelled, skipped and todo (all 0) are omitted. Real projects usually run the same tests with Vitest or Jest (the two most popular JS test runners) — §12.9 shows that setup (not run here).

12.1 What to test, and the Testing Library philosophy

What it is

RTL is a small set of helpers on **DOM Testing Library**: `render` a component into a (simulated) DOM, **query** elements the way a person finds them (role, label, text), fire **events**, and assert on the resulting DOM. Guiding principle: the closer a test is to how people use the software, the more you can trust it.

Test this (what users experience)	Not this (implementation details)
The heading shows "Ada Lovelace" after loading	The component's `user` state variable
Clicking **Add to cart** updates the badge to 2	That `dispatch` was called with `{ type: 'add' }`
The **Submit** button is disabled until terms are accepted	Which Hook or CSS class name is used
An error message appears when the server returns 404 Not Found	The number of re-renders

Like an inspector who tastes the food rather than reading recipe cards: a kitchen reorganisation (a **refactor** — restructuring without changing behaviour) still passes; worse food fails.

Why it matters

- **Refactor-proof**: tests that ignore internals survive renaming state, splitting components, `useState` → `useReducer`, or enabling React Compiler (Chapter 10).
- **Accessibility for free** (UI usable by everyone, including screen-reader and keyboard-only users): finding by role and accessible name only works if the UI is accessible (§12.3 catches an unlabelled icon button).
- **Confidence**: clicking the real button through the real event system catches wiring bugs (e.g. a handler never attached) that a unit test of the handler misses.

Where component tests sit

Kind	Checks	Tools	Speed
Unit	one function (a reducer, a formatter)	`node:test`, Vitest, Jest	fastest
Component / integration	one component or feature, simulated DOM	RTL + jsdom (this chapter)	fast
End-to-end (E2E)	the whole app in a real browser against a real or staged (test-copy) backend	Playwright, Cypress	slowest

A common strategy (the "testing trophy"): most effort in **integration-style component tests**, a few E2E tests for critical flows (sign-up, checkout), unit tests for tricky pure logic (output depends only on inputs, no side effects) — plus static checks that don't run code (TypeScript, Chapter 11; ESLint).

12.2 Your first component test

How it works

1. **Arrange** — `render(<Counter />)` mounts the component into `document.body` (jsdom provides the DOM).
2. **Act** — find an element (`screen.getByRole('button', { name: 'Increment' })`) and interact (`fireEvent.click(...)` or `user-event`, §12.4).
3. **Assert** — check the DOM: text, disabled state, presence/absence.
4. **Clean up** — unmount after each test so tests don't see each other's DOM. Automatic in Jest and Vitest (with `globals: true`); with `node:test` register `afterEach(cleanup)`.

`screen` has every query bound to `document.body`, so you needn't keep `render`'s return value.

`code/01_first_test.jsx`:

```
// imports: tools_dom.mjs (jsdom globals), node:test, node:assert/strict, react, @testing-library/react
function Counter({ initial = 0, step = 1 }) {
  const [count, setCount] = useState(initial);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + step)}>Increment</button>
      <button onClick={() => setCount(initial)} disabled={count === initial}>Reset</button>
    </div>
  );
}

afterEach(cleanup);

test('shows the initial count', () => {
  render(<Counter initial={5} />);
  assert.ok(screen.getByText('Count: 5'));                          // getBy* throws if not found
});

test('increments by step when the button is clicked', () => {
  render(<Counter step={10} />);
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  assert.equal(screen.getByText(/Count:/).textContent, 'Count: 20');
});

test('Reset is disabled until the count changes', () => {
  render(<Counter />);
  const reset = screen.getByRole('button', { name: 'Reset' });
  assert.equal(reset.disabled, true);
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  assert.equal(reset.disabled, false);
  fireEvent.click(reset);
  assert.equal(screen.getByText(/Count:/).textContent, 'Count: 0');
});
```

Output:

```
✔ shows the initial count (13.84825ms)
✔ increments by step when the button is clicked (50.187958ms)
✔ Reset is disabled until the count changes (3.544833ms)
ℹ tests 3
ℹ pass 3
ℹ fail 0
ℹ duration_ms 70.571583
```

What about `act()`?

React batches updates and runs effects asynchronously. `act(() => { ... })` means "apply everything this caused before I look at the DOM". RTL's `render`, `fireEvent`, `user-event` actions and async utilities already wrap themselves in `act`, so you rarely write it — except for **direct** state updates, e.g. calling a Hook's function in `renderHook` (§12.7) or advancing fake timers. The warning *"An update to X inside a test was not wrapped in act(...)"* usually means something finished **after** your assertions — await it with `findBy` / `waitFor` (§12.5).

With jest-dom (in Vitest/Jest)

`@testing-library/jest-dom` adds readable matchers with good failure messages: `expect(reset).toBeDisabled()`, `expect(el).toHaveTextContent('Count: 20')`, `expect(el).toBeInTheDocument()`. This book uses plain `assert` on the same DOM properties (`.disabled`, `.textContent`).

12.3 Queries and their priority

Query variants

The prefix decides **what happens with zero or several matches**, and whether it waits:

Variant	0 matches	1 match	2+ matches	Waits?	Use it to…
`getBy…`	throws	returns element	throws	no	find something that must be there **now**
`queryBy…`	returns `null`	returns element	throws	no	assert something is **absent**
`findBy…`	rejects	resolves element	rejects	**yes** (retries, default 1000 ms)	wait for something to **appear**
`getAllBy…`	throws	array	array	no	find several
`queryAllBy…`	`[]`	array	array	no	count, possibly zero
`findAllBy…`	rejects	array	array	**yes**	wait for several

`findBy…` returns a **Promise** that resolves with the element or rejects after the timeout — `await` it.

Which query to use (priority order)

The higher, the closer to how people (including screen-reader users) find things:

1. **Accessible to everyone**
    1. `ByRole` — by ARIA role (ARIA, Accessible Rich Internet Applications: attributes telling assistive tools what an element is; the *role* is its kind — button, link, heading, textbox, checkbox, list…) and **accessible name** (`{ name: 'Save' }`). Use for almost everything.
    2. `ByLabelText` — form fields by `<label>`.
    3. `ByPlaceholderText` — only if a field truly has no label (a placeholder isn't a label).
    4. `ByText` — non-interactive text.
    5. `ByDisplayValue` — a field by its current value.
2. **Semantic attributes**: `ByAltText` (images, `alt`); `ByTitle` (`title`, not reliably announced by screen readers).
3. **Test IDs**: `ByTestId` — `data-testid="…"`, only when nothing user-visible identifies the element.

The **accessible name** is what assistive technology announces: a button's text, an input's label, `aria-label` / `aria-labelledby`, an image's `alt`. Matchers can be exact strings (`'Save'`), case-insensitive regexes (`/save/i`) or functions.

`code/02_queries.jsx` (component and key tests):

```jsx
function ProductCard({ product, inCart }) {
  return (
    <article aria-labelledby="p-title">
      <img src={product.image} alt={`Photo of ${product.name}`} />
      <h2 id="p-title">{product.name}</h2>
      <p>{product.price.toFixed(2)} EUR</p>
      <label htmlFor="qty">Quantity</label>
      <input id="qty" type="number" defaultValue={1} />
      <input placeholder="Gift message (optional)" />
      <ul aria-label="features">{product.features.map(f => <li key={f}>{f}</li>)}</ul>
      <button>Add to cart</button>
      <button title="Save for later">♡</button>
      {inCart && <p role="status">Already in your cart</p>}
      <span data-testid="sku">{product.sku}</span>
    </article>
  );
}

// priority 1 (role, label, placeholder, text, display value):
screen.getByRole('heading', { name: 'Desk lamp', level: 2 });
screen.getByRole('article', { name: 'Desk lamp' });            // name from aria-labelledby
screen.getByLabelText('Quantity');  screen.getByPlaceholderText(/gift message/i);
screen.getByText('39.50 EUR');      screen.getByDisplayValue('1');
// priority 2 (alt text, title):
screen.getByAltText(/photo of desk lamp/i);  screen.getByTitle('Save for later');
// A button's accessible name comes from its CONTENT first; title is only a fallback. So a screen
// reader announces this icon button as "♡" - the test just revealed an accessibility bug.
screen.getByRole('button', { name: '♡' });                      // found
screen.queryByRole('button', { name: 'Save for later' });       // null
// Fix in the component: <button aria-label="Save for later">♡</button>
// many matches, scoped search:
within(screen.getByRole('list', { name: 'features' })).getAllByRole('listitem');   // LED, Dimmable, USB-C
// absence, then presence after rerender(<ProductCard product={lamp} inCart />):
screen.queryByRole('status');                                   // null -> then getByRole('status')
// ... plus: a failing getBy's message, logRoles, and getByTestId('sku')
```

Output:

```
    error message (first 3 lines):
    | Unable to find an accessible element with the role "button" and name "Buy now"
    |
    | Here are the accessible roles:
    logRoles output:
    | list:
    | Name "tiny":
    | <ul
    |   aria-label="tiny"
    | />
    | ---------------------------------------------------
    | listitem:
    | Name "":
    | <li />
    | ---------------------------------------------------
✔ priority 1: queries everyone can perceive (role, label, placeholder, text, display value) (61.198666ms)
✔ priority 2: semantic attributes (alt text, title) (6.331375ms)
✔ priority 3: test IDs, only when nothing user-visible identifies the element (2.198959ms)
✔ getAllBy for many matches, within() to search inside one element (8.569083ms)
✔ queryBy to assert that something is NOT there (4.234708ms)
✔ a failing getBy explains itself (13.676958ms)
✔ logRoles prints every role and accessible name - handy when writing queries (2.698125ms)
ℹ tests 7
ℹ pass 7
ℹ fail 0
```

Notes on the output

- A failing `getBy` lists **every accessible role and name** (cut to 3 lines here), then the DOM — usually telling you at once whether the name is wrong or the element is missing.

- `logRoles(container)` prints the role list on demand; `screen.debug()` prints the DOM. For writing tests, not leaving in them.
- Common roles: `button`, `link`, `heading` (with `level`), `textbox`, `checkbox`, `radio`, `combobox` (a `<select>`), `spinbutton` (number input), `list` / `listitem`, `table` / `row` / `cell`, `dialog`, `alert`, `status`, `img`, `navigation`, `main`.
- The icon button had a `title` but was announced as "♡" — role queries make such bugs visible.

Common mistakes

Mistake	Better
`container.querySelector('.btn-primary')`	`screen.getByRole('button', { name: 'Save' })`
`getByTestId` everywhere	role/label/text first; test IDs last resort
`expect(queryByText('x')).toBeTruthy()` to check presence	`getByText` (throws with a helpful message)
`getByText` to assert absence (it throws)	`queryByText(...) === null`
`getBy` for things that appear later	`findBy` (§12.5)
Exact long text that breaks on copy edits	regex for the stable part: `/order confirmed/i`

12.4 `user-event` — interacting like a user

What it is

`fireEvent` dispatches **one** DOM event. A real user typing "hi" produces focus, click, and per key `keydown`, `keypress`, `input`, `keyup`; a real click on a disabled button does nothing; Tab moves focus in document order. `@testing-library/user-event` simulates those full interactions, exercising the same paths as real users (including bugs that only appear with the full sequence, like a `keydown` handler).

```
import userEvent from '@testing-library/user-event';
const user = userEvent.setup();            // v14 API: create a session, then await each action
await user.type(input, 'Ada');
await user.click(button);
await user.selectOptions(select, 'pro');
await user.keyboard('{Enter}');            // special keys in braces: {Enter} {Escape} {Backspace} {Shift>}...
await user.tab();
await user.clear(input);
await user.upload(fileInput, file);
await user.hover(element);
```

All methods are **async** — always `await` them.

`code/03_user_event.jsx` (abridged; `SignupForm` has a Name input, a Plan select, a terms checkbox, and a submit button disabled until terms and name are set):

```
test('fireEvent vs userEvent: which DOM events does "typing" produce?', async () => {
  render(<input aria-label="box" />);
  const box = screen.getByLabelText('box');
  const seen = [];
  for (const type of ['focus', 'keydown', 'keypress', 'input', 'keyup', 'click'])
    box.addEventListener(type, () => seen.push(type));
  fireEvent.change(box, { target: { value: 'hi' } });
  console.log('    fireEvent.change   ->', seen.join(', ') || '(no listened-to events; only a synthetic change)');
  seen.length = 0; box.value = '';
  const user = userEvent.setup();
  await user.type(box, 'hi');
  console.log('    user.type(box, hi) ->', seen.join(', '));
});

test('fills in and submits the whole form like a user', async () => {
  const user = userEvent.setup();
  const submitted = [];
  render(<SignupForm onSubmit={data => submitted.push(data)} />);
  const button = screen.getByRole('button', { name: 'Create account' });
  assert.equal(button.disabled, true);
  await user.type(screen.getByRole('textbox', { name: 'Name' }), 'Ada');
  await user.selectOptions(screen.getByRole('combobox', { name: 'Plan' }), 'pro');
  await user.click(screen.getByRole('checkbox', { name: 'I accept the terms' }));
  assert.equal(button.disabled, false);
  await user.click(button);
  assert.deepEqual(submitted, [{ name: 'Ada', plan: 'pro', terms: true }]);
});

test('keyboard only: Tab moves focus in DOM order, Enter submits', async () => {
  // user.tab() focuses Name; user.keyboard('Grace'); two more tabs reach the checkbox;
  // user.keyboard(' ') toggles it (Space); '{Enter}' in the text field submits the form
});

test('clicking a disabled button does nothing', async () => {
  const user = userEvent.setup();
  let clicks = 0;
  render(<button disabled onClick={() => clicks++}>Pay</button>);
  await user.click(screen.getByRole('button', { name: 'Pay' }));
  assert.equal(clicks, 0);
});
```

Output:

```
    fireEvent.change   -> (no listened-to events; only a synthetic change)
    user.type(box, hi) -> focus, click, keydown, keypress, input, keyup, keydown, keypress, input, keyup
✔ fireEvent vs userEvent: which DOM events does "typing" produce? (72.232958ms)
✔ fills in and submits the whole form like a user (55.993083ms)
✔ keyboard only: Tab moves focus in DOM order, Enter submits (49.1405ms)
✔ clicking a disabled button does nothing (7.998ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

`fireEvent` VS `user-event`

	`fireEvent`	`user-event`
What it does	dispatches exactly one event	simulates the full interaction (focus, pointer, keyboard, input)
Sync/async	sync	async (`await`)
Respects disabled, `pointer-events: none`, focus order	no	yes
When to use	events user-event doesn't model (`scroll` , custom events), very low-level tests	**default** for anything a user does

The keyboard test doubles as an accessibility check: the form works without a mouse.

12.5 Async tests

The problem it solves

Right after `render`, loaded data isn't there, so `getBy` fails. **Wait for the UI**, not a fixed time — `await sleep(500)` is slow and **flaky** (passes or fails with no code change, e.g. fine locally, failing on a busy CI server).

Tools

Tool	Waits for	Typical use
`await screen.findByRole(...)`	an element to appear (retries a `getBy`)	data loaded, dialog opened
`await waitFor(() => { ...assertions... })`	a callback to stop throwing	text change, a mock being called
`await waitForElementToBeRemoved(() => screen.queryByText('Loading…'))`	an element to disappear	spinners, closing dialogs

All retry every 50 ms (default) until success or timeout (default 1000 ms), wrapping updates in `act`.

`code/04_async.jsx` (abridged):

```
const api = {
  loadOrders: () => new Promise(r => setTimeout(() => r([{ id: 1, total: 20 }, { id: 2, total: 35 }]), 50)),
  save: () => new Promise(r => setTimeout(r, 30)),
};
function Orders() {
  const [orders, setOrders] = useState(null);
  const [saved, setSaved] = useState(false);
  useEffect(() => { api.loadOrders().then(setOrders); }, []);
  if (!orders) return <p>Loading orders…</p>;
  return (<>
    <ul>{orders.map(o => <li key={o.id}>Order {o.id}: {o.total} EUR</li>)}</ul>
    <button onClick={async () => { await api.save(); setSaved(true); }}>Save</button>
    {saved && <p role="status">Saved!</p>}
  </>);
}

test('findBy waits (retries up to 1000 ms by default) until the element appears', async () => {
  render(<Orders />);
  const items = await screen.findAllByRole('listitem');
  assert.deepEqual(items.map(li => li.textContent), ['Order 1: 20 EUR', 'Order 2: 35 EUR']);
});
// waitForElementToBeRemoved(() => screen.queryByText('Loading orders…')) then 2 listitems
test('waitFor retries any assertion until it passes', async () => {
  const user = userEvent.setup();
  render(<Orders />);
  await user.click(await screen.findByRole('button', { name: 'Save' }));
  await waitFor(() => assert.equal(screen.getByRole('status').textContent, 'Saved!'));
});
test('findBy gives up after its timeout with a clear error', async () => {
  render(<Orders />);
  await assert.rejects(screen.findByText('Order 99', {}, { timeout: 200 }), /Unable to find an element with the
text: Order 99/);
});
```

Output:

```
✔ right after render only the loading state exists (14.355875ms)
✔ findBy waits (retries up to 1000 ms by default) until the element appears (98.487708ms)
✔ waitForElementToBeRemoved waits for the spinner to go away (55.889ms)
✔ waitFor retries any assertion until it passes (101.515625ms)
    rejected after ~200 ms
✔ findBy gives up after its timeout with a clear error (202.66725ms)
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

Rules for `waitFor`

- **Only assertions** inside — it may run many times; actions go outside.
- One thing to wait for per `waitFor`; assert the rest normally.
- Prefer `findBy` for elements (reads better, better errors); never wrap `findBy` in `waitFor`.

Common mistakes

- Missing `await` before `findBy` / `waitFor` / `user.*` — the test ends before the UI updates, passes by accident, and prints an `act(...)` warning.
- Arbitrary `sleep`s — wait for a **condition**. For truly time-based code — debounce (act once input has paused), polling (re-asking the server every few seconds) — use fake timers (§12.7).

12.6 Mocking `fetch` (and when to use MSW)

What it is

A **mock** replaces a real dependency with a controllable fake. Tests shouldn't hit a real server (slow, needs network, data changes). Mocking `fetch` lets each test choose the response — success, an HTTP error (a 4xx/5xx status; see System Design Fundamentals), a network failure, a slow reply — and check the UI for each.

With `node:test`, `mock.method(globalThis, 'fetch', impl)` replaces the global `fetch` and records calls; `mock.restoreAll()` restores it. (Vitest: `vi.spyOn(globalThis, 'fetch').mockResolvedValue(...)`; Jest: `jest.spyOn(global, 'fetch')`.) Node 18+ has `Response` built in, so fakes return real `Response` objects.

`code/05_mocking_fetch.jsx` (abridged):

```
function UserProfile({ userId }) {
  const [state, setState] = useState({ status: 'loading' });
  useEffect(() => {
    const controller = new AbortController();
    setState({ status: 'loading' });
    fetch(`/api/users/${userId}`, { signal: controller.signal })
      .then(res => { if (!res.ok) throw new Error(`Server said ${res.status}`); return res.json(); })
      .then(user => setState({ status: 'done', user }))
      .catch(err => { if (err.name !== 'AbortError') setState({ status: 'error', message: err.message }); });
    return () => controller.abort();                  // cancel if userId changes or we unmount
  }, [userId]);
  if (state.status === 'loading') return <p>Loading…</p>;
  if (state.status === 'error') return <p role="alert">Could not load user: {state.message}</p>;
  return <h1>{state.user.name}</h1>;
}

const json = (body, status = 200) => new Response(JSON.stringify(body), { status, headers: { 'Content-Type':
'application/json' } });
afterEach(() => { cleanup(); mock.restoreAll(); });    // put the real fetch back after every test

test('shows the user returned by the server, and calls the right URL', async () => {
  const fetchMock = mock.method(globalThis, 'fetch', async () => json({ id: 7, name: 'Ada Lovelace' }));
  render(<UserProfile userId={7} />);
  assert.equal((await screen.findByRole('heading')).textContent, 'Ada Lovelace');
  assert.equal(fetchMock.mock.calls[0].arguments[0], '/api/users/7');
});
// HTTP error: fetch resolves json({...}, 404) -> alert "Could not load user: Server said 404" (fetch does not
reject on 404)
// network error: fetch throws new TypeError('Failed to fetch') -> alert matches /Failed to fetch/
test('a slow old response cannot overwrite a newer one (race condition)', async () => {
  // user 1 answers after 100 ms, user 2 after 10 ms; honour the abort signal like real fetch does
  const delays = { '/api/users/1': 100, '/api/users/2': 10 };
  mock.method(globalThis, 'fetch', (url, { signal }) => new Promise((resolve, reject) => {
    const t = setTimeout(() => resolve(json({ name: `User from ${url}` })), delays[url]);
    signal.addEventListener('abort', () => { clearTimeout(t); reject(new DOMException('aborted', 'AbortError')); });
}));
  const { rerender } = render(<UserProfile userId={1} />);
  rerender(<UserProfile userId={2} />);                  // user switches before the first answer
  assert.equal((await screen.findByRole('heading')).textContent, 'User from /api/users/2');
  await new Promise(r => setTimeout(r, 150));              // wait past the slow response
  assert.equal(screen.getByRole('heading').textContent, 'User from /api/users/2');
});
// ... plus a test that clicks "Next user" and waits for the second request's result
```

Output:

```
✔ shows the user returned by the server, and calls the right URL (67.915ms)
✔ shows an error for an HTTP error status (fetch does not reject on 404) (6.110375ms)
✔ shows an error when the network fails (4.473667ms)
✔ a slow old response cannot overwrite a newer one (race condition) (168.044584ms)
✔ mock only the network call, then drive the UI as a user (20.320625ms)
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

Does the race test really test something? (A **race condition**: the result depends on which overlapping operation finishes first.) A test that can't fail is worthless. Removing the `controller.abort()` cleanup (in a temporary copy) makes it fail:

```
✖ a slow old response cannot overwrite a newer one (race condition) (168.159708ms)
  + actual - expected
    actual: 'User from /api/users/1',
    expected: 'User from /api/users/2',
```

Checking that a test fails on broken code is a good habit. (Race and `AbortController` fix: Chapter 5.) The last test shows the preferred style: mock **only the network boundary**, drive the real UI with `user-event`.

Mock Service Worker (MSW) — the usual choice in real projects

MSW intercepts requests at the network level (in the browser via a Service Worker — a background script between page and network) instead of replacing `fetch`, so it works for `fetch`, `axios`, TanStack Query and anything else, and the same handlers (functions choosing the fake response per URL) can power the dev server. Sketch (MSW 2 — not run here):

```
// test/server.js
import { setupServer } from 'msw/node';
import { http, HttpResponse } from 'msw';
export const server = setupServer(
  http.get('/api/users/:id', ({ params }) => HttpResponse.json({ id: params.id, name: 'Ada Lovelace' })),
);
// setupTests.js
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());       // undo per-test overrides
afterAll(() => server.close());
// in one test: server.use(http.get('/api/users/:id', () => new HttpResponse(null, { status: 404 })));
```

Approach	Good for	Drawback
Replace `fetch` (`mock.method`, `vi.spyOn`)	small projects, few tests, no dependency	tied to `fetch`; response setup repeated per test
MSW	real apps; any HTTP client; shared handlers for tests, Storybook (components in isolation) and dev	one more library and setup file
Mock your own API module (`vi.mock('./api')`)	quick isolation	skips your fetch/parse/error code — lower confidence

Components using **TanStack Query** need a `QueryClientProvider` in the wrapper — a **new** `QueryClient` per test with `retry: false`, so tests don't share cache and failures surface immediately (Chapter 9; providers in §12.8).

12.7 Testing custom Hooks

Two approaches

1. **Through a component** using the Hook as the app does — most realistic, often enough.
2. **`renderHook`** for reusable Hooks with a clear API (`useCounter`, `useDebounce`): renders a hidden component calling your Hook and exposes the latest return as `result.current`.

Hooks can't be called outside a component (Chapter 7), so both render something.

`renderHook`, `act`, and `rerender`

- `const { result, rerender, unmount } = renderHook(() => useX(args), { initialProps, wrapper })`.
- Read `result.current` **after** each update — it's replaced every render.
- Wrap state-updating calls in **`act(() => ...)`** (no RTL helper is doing it).
- `rerender(newProps)` re-renders the same Hook instance with new arguments (`initialProps` + a callback taking props).
- Context-dependent Hooks take a `wrapper` as in §12.8.

Fake timers

Time-based Hooks (debounce, throttle — at most once per interval — polling, timeouts) shouldn't make tests wait. **Fake timers** replace `setTimeout` & co. with manually advanced versions: `node:test` `mock.timers.enable({ apis: ['setTimeout'] })` / `mock.timers.tick(ms)`; Vitest `vi.useFakeTimers()` / `vi.advanceTimersByTime(ms)`; Jest `jest.useFakeTimers()` / `jest.advanceTimersByTime(ms)`. Advance inside `act`.

`code/06_testing_hooks.jsx` (abridged):

```
function useCounter(initial = 0, { min = -Infinity, max = Infinity } = {}) {
  const [count, setCount] = useState(initial);
  const increment = useCallback(() => setCount(c => Math.min(max, c + 1)), [max]);
  const decrement = useCallback(() => setCount(c => Math.max(min, c - 1)), [min]);
  const reset = useCallback(() => setCount(initial), [initial]);
  return { count, increment, decrement, reset };
}
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}
afterEach(() => { cleanup(); mock.timers.reset(); });

test('useCounter: increments, respects max, resets', () => {
  const { result } = renderHook(() => useCounter(8, { max: 10 }));
  act(() => result.current.increment());              // state updates must be wrapped in act()
  act(() => result.current.increment());
  act(() => result.current.increment());              // would be 11, capped at 10
  assert.equal(result.current.count, 10);
  act(() => result.current.reset());
  assert.equal(result.current.count, 8);
});
test('useCounter: rerender with new arguments', () => {
  const { result, rerender } = renderHook(({ max }) => useCounter(0, { max }), { initialProps: { max: 1 } });
  act(() => { result.current.increment(); result.current.increment(); });
  assert.equal(result.current.count, 1);
  rerender({ max: 5 });                               // same Hook instance, new props
  act(() => result.current.increment());
  assert.equal(result.current.count, 2);
});
test('useDebounce: only the last value within the delay gets through', () => {
  mock.timers.enable({ apis: ['setTimeout'] });
  const { result, rerender } = renderHook(({ v }) => useDebounce(v, 300), { initialProps: { v: 'r' } });
  rerender({ v: 're' });
  act(() => mock.timers.tick(200));
  rerender({ v: 'rea' });                             // restarts the 300 ms timer
  act(() => mock.timers.tick(299));
  assert.equal(result.current, 'r');                  // nothing has settled yet
  act(() => mock.timers.tick(1));
  assert.equal(result.current, 'rea');                // 're' was skipped entirely
});
// + 'useCounter through a component': a <Stepper> with -/+ buttons, clicked with user-event, cannot go below min
```

Output:

```
✔ useCounter: increments, respects max, resets (7.892375ms)
✔ useCounter: rerender with new arguments (1.192583ms)
✔ useDebounce: only the last value within the delay gets through (1.022ms)
✔ useCounter through a component: cannot go below min (79.11725ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

The debounce test covers 800 ms of "time" in about 1 ms.

Common mistakes

- `const { count } = result.current` once, expecting it to update — re-read `result.current.count` after each `act`.
- Calling the Hook directly outside a component → "Invalid hook call".
- Leaving fake timers on — reset in `afterEach`. `user-event` waits between actions with timers, so with fake timers in Vitest/Jest use `userEvent.setup({ advanceTimers: vi.advanceTimersByTime })` (or `jest.advanceTimersByTime`).

- Importing `renderHook` from `@testing-library/react-hooks` — that package is for React 17 and older; since RTL 13.1 it's in `@testing-library/react`.

12.8 Providers, routers and custom render

The problem it solves

Real components depend on providers: context (theme, auth, cart), a router, a TanStack Query client, a Redux store. Rendered alone they crash; repeating the provider tree per test is noisy. So projects write one **custom** `render` wrapping the UI via RTL's `wrapper` option.

`code/07_providers_and_router.jsx` (abridged):

```
export function cartReducer(state, action) {            // pure function: unit-test it directly
  switch (action.type) {
    case 'add': return { ...state, [action.sku]: (state[action.sku] ?? 0) + 1 };
    case 'clear': return {};
    default: throw new Error(`unknown action ${action.type}`);
  }
}
// CartProvider (useReducer + CartContext); CartBadge reads the cart; ProductPage reads :sku and dispatches 'add';
// App = <nav> with <Link>s and <CartBadge/> + <Routes> for "/" and "/p/:sku".

function renderWithProviders(ui, { route = '/', cart = {} } = {}) {
  const Wrapper = ({ children }) => (
    <MemoryRouter initialEntries={[route]}>            {/* a router that keeps history in memory */}
      <CartProvider initial={cart}>{children}</CartProvider>
    </MemoryRouter>
  );
  return { user: userEvent.setup(), ...render(ui, { wrapper: Wrapper }) };
}

test('reducer: plain unit test, no React needed', () => {
  assert.deepEqual(cartReducer({ MUG: 1 }, { type: 'add', sku: 'MUG' }), { MUG: 2 });
  assert.throws(() => cartReducer({}, { type: 'oops' }), /unknown action oops/);
});
test('starts on a given route with a pre-filled cart', () => {
  renderWithProviders(<App />, { route: '/p/LAMP', cart: { MUG: 2 } });
  assert.equal(screen.getByRole('heading').textContent, 'Product LAMP');
  assert.equal(screen.getByLabelText('items in cart').textContent, '2');
});
test('navigates with a link, then adds to cart', async () => {
  const { user } = renderWithProviders(<App />);
  await user.click(screen.getByRole('link', { name: 'Mug' }));
  await user.click(screen.getByRole('button', { name: 'Add to cart' }));
  await user.click(screen.getByRole('button', { name: 'Add to cart' }));
  assert.equal(screen.getByLabelText('items in cart').textContent, '2');
});
test('without the provider the component crashes - which is why the wrapper exists', () => {
  assert.throws(() => render(<CartBadge />), /Cannot destructure property 'cart'/);   // console.error silenced
});
```

Output:

```
✔ reducer: plain unit test, no React needed (0.762041ms)
✔ starts on a given route with a pre-filled cart (58.361708ms)
✔ navigates with a link, then adds to cart (45.55225ms)
✔ without the provider the component crashes - which is why the wrapper exists (2.687916ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

How it works

1. `render(ui, { wrapper: Wrapper })` renders `<Wrapper>{ui}</Wrapper>`; `rerender` keeps the wrapper.

2. `MemoryRouter` (from `react-router`) keeps history in memory; `initialEntries` sets the starting URL. For data-router apps (`createBrowserRouter`, loaders/actions), use `createMemoryRouter(routes, { initialEntries })` + `<RouterProvider>`.
3. Options like `route` and `cart` start each test from a state **through the public API** (provider props), not by poking internals.
4. Put `renderWithProviders` in a shared `test-utils` file. With Redux Toolkit create a **fresh store per test**; with TanStack Query a **fresh** `QueryClient` — shared instances leak state between tests.

Unit tests still have a place

The pure reducer gets a plain, very fast unit test. Test pure logic (reducers, formatters, validation schemas) directly; test **components** through the UI.

12.9 Setting up Vitest or Jest in a real project

Real projects usually use **Vitest** (natural for Vite; Jest-compatible API) or **Jest**. Component tests look the same — only imports, `expect` and mock helpers differ. Standard setups, **not run in this book**.

Vitest (Vite projects)

```
npm install -D vitest jsdom @testing-library/react @testing-library/dom @testing-library/user-event @testing-library/jest-dom
```

```
// vite.config.js (or vitest.config.js)
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',          // simulated browser DOM (alternative: 'happy-dom')
    globals: true,                 // describe/test/expect/afterEach as globals -> RTL auto-cleanup works
    setupFiles: './src/setupTests.js',
  },
});
```

```
// src/setupTests.js
import '@testing-library/jest-dom/vitest';   // adds toBeInTheDocument(), toBeDisabled(), toHaveTextContent()...
```

```
// package.json
{ "scripts": { "test": "vitest", "test:run": "vitest run", "coverage": "vitest run --coverage" } }
```

```
// Counter.test.jsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Counter } from './Counter';

test('increments', async () => {
  const user = userEvent.setup();
  render(<Counter />);
  await user.click(screen.getByRole('button', { name: /increment/i }));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

With TypeScript add `"types": ["vitest/globals", "@testing-library/jest-dom"]` to `tsconfig.json`. Without `globals: true`, import `test`, `expect`, `afterEach` from `vitest` and call `afterEach(cleanup)` yourself.

Jest

```
npm install -D jest jest-environment-jsdom babel-jest @babel/preset-env @babel/preset-react \
  @testing-library/react @testing-library/dom @testing-library/user-event @testing-library/jest-dom
```

```
// jest.config.js
module.exports = {
  testEnvironment: 'jsdom',                       // needs jest-environment-jsdom (separate since Jest 28)
  setupFilesAfterEnv: ['<rootDir>/jest.setup.js'], // runs after the test framework is installed
  moduleNameMapper: { '\\.(css|less|svg)$': '<rootDir>/test/fileMock.js' },  // stub non-JS imports
};
// jest.setup.js
import '@testing-library/jest-dom';
// babel.config.js
module.exports = { presets: ['@babel/preset-env', ['@babel/preset-react', { runtime: 'automatic' }]] };
```

TypeScript with Jest needs `ts-jest` or Babel's TypeScript preset. Vitest reuses Vite's transform pipeline, so needs none of this — one reason new Vite projects choose it.

Useful extras

Tool	For
`@testing-library/jest-dom`	readable DOM matchers
MSW	network mocking (§12.6)
`vitest --coverage` / `jest --coverage`	coverage (which lines ran) — a guide to untested code, not a goal
`eslint-plugin-testing-library`, `eslint-plugin-jest-dom`	lint rules catching misuse (`getBy` inside `waitFor`, missing `await`)
Vitest Browser Mode, Playwright component testing	component tests in a real browser when jsdom isn't enough (layout, real CSS, canvas)
Playwright / Cypress	end-to-end tests

What jsdom can't do

jsdom implements DOM and events but **no layout or painting**: sizes/positions are 0, `scrollTop` changes only when you set it (as in Chapter 10's virtualization demo), CSS media queries don't apply, and `IntersectionObserver` (element scrolled into view), `ResizeObserver` (size changes) and `matchMedia` (media query from JS) are absent by default and must be mocked. Real-rendering behaviour belongs in browser-based or E2E tests.

12.10 Good tests, bad tests, and the bigger picture

What makes a good component test

1. **Named after behaviour**: "shows an error when the server returns 404", not "test 3" or "sets error state".
2. **Arrange–act–assert**, one behaviour per test (several asserts fine if one behaviour).
3. **Public interface**: props in, DOM out, user events between, network mocked at the boundary.
4. **Independent**: no shared DOM, mocks, stores or caches; any order works.
5. **Deterministic**: no real network, real time or randomness (or seeded — repeatable from a fixed start value); waits for conditions, never sleeps.
6. **Fails for the right reason**: break the feature → red with a clear message (§12.6's race test).

Brittle tests to avoid

A **brittle** test breaks on harmless code changes even though the feature works.

Brittle	Why	Instead
Asserting on state, props, or Hook internals	breaks on refactor, passes when UI is broken	assert on rendered output
`container.querySelector('.css-1x2y3z')`	class names are implementation details	role/label/text queries
Large **snapshot** tests (saved rendered output; fails on any change) of whole components	any markup change fails; reviewers approve new snapshots unread	explicit assertions; small inline snapshots at most
Mocking child components	no longer checks real integration	render the real tree; mock only network or truly heavy leaf modules
Asserting a handler was called with internal arguments	couples to wiring	assert the handler's visible effect
Testing third-party libraries	not your code	test your usage via the UI

Real-world examples

- Checkout form: "button disabled until valid", "shows field errors", "submits the right payload" (checked at the mocked network), "shows server error", plus one E2E test of the full purchase.
- Design-system `Dialog`: role `dialog`, labelled by its title, focus moves inside on open, Escape closes, focus returns to the trigger — all testable with RTL + user-event, and each an accessibility requirement (Chapter 14).

In the interview

30-second answer: "React Testing Library with Vitest or Jest. Tests render the component, find elements the way users do — role and accessible name first — interact with user-event, and assert on the DOM, not state or internals. Async UI is awaited with `findBy` / `waitFor`, never sleeps. The network is mocked at the boundary, ideally with MSW, covering success, errors and races. Hooks are tested through components or `renderHook` with `act`; provider-dependent components use a custom render with a wrapper. Playwright E2E covers the few critical flows."

- *`getBy` vs `queryBy` vs `findBy`?* — `getBy` throws if missing (must exist now), `queryBy` returns `null` (assert absence), `findBy` returns a retrying promise (async appearance).
- *Why prefer `getByRole`?* — Matches how users and assistive technology find elements, doubles as an accessibility check, survives markup refactors.
- *What does `act` do and when do you need it?* — Flushes React updates and effects before assertions; RTL helpers wrap it, so only for direct updates (`renderHook` calls, advancing fake timers).
- *How do you test a component that fetches?* — Mock the network (MSW or a `fetch` spy), `await findBy…` the loaded UI, test error and loading states too.

Self-check

1. Query for a "Save" button? → `screen.getByRole('button', { name: 'Save' })`.
2. Assert an error is **not** shown? → `assert.equal(screen.queryByRole('alert'), null)` (or `expect(...).not.toBeInTheDocument()`).
3. Why is `await new Promise(r => setTimeout(r, 500))` bad for waiting on data? → Slow and flaky; wait for the condition with `findBy` / `waitFor`.
4. Test `useDebounce` without waiting? → Fake timers advanced inside `act`.
5. Why a fresh store/query client per test? → Shared instances leak state and make tests order-dependent.
6. Why must `node:test` files call `afterEach(cleanup)`? → RTL auto-cleans only when a global `afterEach` exists (Jest, Vitest with globals).

Chapter 12 TL;DR

Idea	Remember
Philosophy	Test what users see and do; ignore implementation details
Basic test	`render` → `screen` query → interact → assert → `cleanup` (automatic in Jest/Vitest)
Query priority	Role (+ name) → label → placeholder → text → display value → alt → title → test ID
Variants	`get` now / `query` maybe-absent / `find` wait; `All` for many; `within` to scope
Interactions	`const user = userEvent.setup(); await user.click/type/selectOptions/keyboard/tab(...)`; prefer over `fireEvent`
Async	`findBy`, `waitFor(() => assertion)`, `waitForElementToBeRemoved`; never sleep
Network	Mock at the boundary (MSW, or a `fetch` spy returning `Response`s); cover success, HTTP error, network error, races; restore mocks
Hooks	Through a component, or `renderHook` + `act` + `result.current`, `rerender` for new args; fake timers for time
Providers	Custom render with `wrapper`: `MemoryRouter`, context, fresh store/query client per test; unit-test pure functions directly
Setup	Vitest (`jsdom`, `globals`, jest-dom setup) or Jest (`jest-environment-jsdom`, Babel/TS, `setupFilesAfterEnv`)
Quality	Behaviour-named, independent, deterministic, fails for the right reason; few snapshots; E2E for critical paths

Next: Chapter 13 — Server Rendering and Frameworks.